

Tanstack-Query-v5

<https://tanstack.com/query/latest>

https://medium.com/@you_0/react-query-is-all-you-need-25af68f44d4f

Server State vs. Client State

Server State: This is data coming from backend or any external API. Because it lives outside your app, it's not always immediately up to date.

Client State: This is the data generated by your application. It's data local to your app's session and isn't stored externally. For instance, UI filters or the current page number in pagination are purely client state because they exist only inside your app's memory.

Where React Query Fits vs. Client-State Libraries

A good rule of thumb:

If the data changes on the server side (like a user's profile that another admin might update), React Query is perfect.

If it's something only your UI cares about (like a local setting), a client-state library is often your best bet.

React Query:

It's built specifically for asynchronous server state — fetching it, caching it, invalidating it, and keeping it up to date automatically.

Client-State Libraries (Redux, MobX, etc.): These libraries are designed to manage synchronous data that your app generates or relies on internally. They don't inherently handle things like data-fetching logic, caching, or background refetching.

Comparison: Without React Query vs. With React Query

Without React Query (using just `useEffect` and `useState`):

```
const [isLoading, setIsLoading] = useState(false);
const [error, setError] = useState(null);
const [data, setData] = useState(null);

useEffect(() => {
  let ignore = false;
  setIsLoading(true);

  fetch(`${BASE_URL}?page=${filters.page}&limit=${filters.limit}`)
    .then((response) => {
      if (!response.ok) {
        throw new Error("Failed to fetch users");
      }
      return response.json();
    })
    .then((data) => {
      if (!ignore) {
        setData(data);
        setIsLoading(false);
      }
    })
    .catch((error) => {
      setError(error);
      setIsLoading(false);
    })
    .finally(() => {
      setIsLoading(false);
    });

  return () => {
    ignore = true;
  };
}, [filters]);
```

With React Query (using useQuery):

```
const { data, isLoading, error } = useQuery({
  queryKey: ["users", "list", filters.page, filters.limit],
  queryFn: () =>
    fetch(`${BASE_URL}?page=${filters.page}&limit=${filters.limit}`).then(
      (response) => {
        if (!response.ok) {
          throw new Error("Failed to fetch users");
        }
        return response.json();
      }
    ),
});
```

Core React Query Features

useQuery is your go-to for reading data. It:

1. Takes a queryKey (usually an array) that uniquely identifies the data.
2. Takes a queryFn (an async function that returns the data).
3. Returns an object containing data, isLoading, error, and other statuses.

useMutation is for creating, updating, or deleting data on the server. It:

1. Receives a mutationFn (an async function to update the server).
2. Returns mutate (a function you call to trigger the mutation), plus isPending, error, etc.

// good

<https://www.it-labs.com/stop-using-redux-for-server-state-why-tanstack-query-is-the-better-choice-in-2025/>

When Should You Still Use Redux? (Client-State Libraries)

...

Authentication Flow and Global Tokens

If you need to:

- Store authentication tokens.
- Keep track of user sessions.
- Guard routes based on login state.

Then having a centralized Redux store still makes a lot of sense.

// good project

<https://profy.dev/article/react-query-usemutation>